Ejercicio Práctico - Node + pg

Actividad

Node + `pg`: conexión por configuración y por *connection string* (pool) con endpoints GET

Introducción / Objetivo

Implementar un servidor en **Node.js** que use el paquete `pg` para conectarse a **PostgreSQL** mediante dos enfoques (configuración de campos y *connection string*), utilizando **pool de conexiones**, exponiendo **endpoints GET** y mostrando los datos en un **frontend** simple. Se busca practicar conexión, consulta y manejo de errores.

Descripción de la actividad

1) El paquete `pg` — Conexión **por configuración** (pool)

PARTE 1 (backend):

- En un servidor web creado en Node, establezca una conexión a la BD PostgreSQL (creada en el Módulo 5) **mediante configuración** (`host`, `port`, `user`, `password`, `database`) y utilice un **pool de conexiones**.
- Exponga **GET** (por ejemplo, `GET /finanzas`) que devuelva **todos los registros** de la tabla `finanzas_personales`.
- Si la tabla no existe, **créela y pueble** algunos registros.
- Aplique **manejo de errores** (respuestas con código y mensaje).

PARTE 2 (frontend):

- Cree una página que consuma el endpoint anterior y muestre la información en **formato de tabla**, incluyendo manejo de errores en la interfaz.

---

2) El paquete `pg` — Conexión **por *connection string*** (pool)

PARTE 1 (backend):

- En un servidor web creado en Node, establezca la conexión a la misma BD **mediante** `connectionString` (ej.: `postgres://user:pass@host:port/db`) y utilice **pool de conexiones**.
- Exponga **GET** (por ejemplo, `GET /clientes`) que devuelva **todos los registros** de la tabla `clientes`.
- Si la tabla no existe, **créela y pueble** algunos registros.
- Aplique **manejo de errores**.

PARTE 2 (frontend):

- Cree una página que consuma el endpoint anterior y muestre la información en **formato de lista**, con manejo de errores.

Guías técnicas mínimas (opcionales)

- **Pool por configuración:**
- `import { Pool } from 'pg';`
- `const pool = new Pool({ host, port, user, password, database });`
- `const { rows } = await pool.query('SELECT * FROM finanzas_personales');`
- **Pool por `connectionString`:**
- `const pool = new Pool({ connectionString: process.env.DATABASE_URL });`
- `const { rows } = await pool.query('SELECT * FROM clientes');`
- Responda en **JSON** con `Content-Type: application/json` y códigos apropiados (`200`, `500`).
- En el frontend, capture errores de `fetch` y muestre un mensaje visible al usuario.

Forma de entrega

Suba un **.zip** con:

- **Backend** (dos enfoques: configuración y `connectionString`) y **Frontend** (tabla y lista).
- Scripts **SQL** usados para crear/poblar `finanzas_personales` y `clientes` (si fue necesario).
- **Pantallazos** de cada vista funcionando.

Tiempo aproximado

90–120 minutos.

Fecha de entrega

Será definida y comunicada por el profesor en la plataforma.